CASCADE: Autonomous Orbital Collision Avoidance for Congested LEO

National Space Hackathon 2026 — IIT Delhi

Team Space_Invaders

https://github.com/Aqwerty321/PROJECT-B.O.N.K

March 2026

Abstract

Low Earth Orbit is entering an era of unprecedented congestion: over 27,000 tracked objects share an increasingly crowded shell, and every new mega-constellation launch raises the probability of a cascading Kessler event. This report presents **CASCADE** (**C**ollision **A**voidance **S**ystem for **C**onstellation **A**utomation, **D**etection & **E**vasion), a fully autonomous, deterministic constellation manager built from the ground up to address all six evaluation axes of the National Space Hackathon 2026 problem statement—safety, fuel efficiency, constellation uptime, algorithmic speed, visualization quality, and code integrity. CASCADE ingests high-frequency ECI J2000 telemetry, screens conjunctions through a mathematically conservative multi-tier filter cascade with a provable zero-false-negative guarantee, and autonomously plans and schedules evasion and recovery burns under realistic propulsion, cooldown, and line-of-sight constraints across a six-station global ground network. Under the reference scenario (50 satellites, 10,000 debris), the C++20 engine completes a full simulation tick—propagation, screening, CDM generation, maneuver planning, and recovery scheduling—in **13.3 ms** mean wall-clock time. Validated against a live Space-Track OMM catalog of **27,191** LEO objects, CASCADE maintains zero missed conjunctions, sub-kilometer station-keeping accuracy, and Tsiolkovsky-consistent fuel accounting across all test families. The system ships as a single Docker image (`ubuntu:22.04`, port 8000) with a React 19 / TypeScript mission-console frontend built on Canvas 2D and Three.js WebGL, backed by 8 core safety gates integrated into the Docker build and GitHub Actions workflow.

1. Introduction

The accelerating growth of resident space objects in LEO—driven by commercial mega-constellations, defunct payloads, and fragmentation debris—has made autonomous collision avoidance a critical capability for any responsible constellation operator. The National Space Hackathon 2026 distils this real-world challenge into a well-defined engineering problem: build an Autonomous Constellation Manager (ACM) that can (i) ingest high-frequency ECI J2000 state vectors, (ii) predict conjunctions up to 24 hours ahead without $O(N^2)$ brute-force enumeration, (iii) autonomously compute, validate, and schedule evasion and station-recovery maneuvers, and (iv) present the full operational picture through an interactive mission dashboard.

From the outset, our team adopted a design philosophy rooted in three principles that guided every architectural decision:

- **Mathematical safety guarantees.** A single missed conjunction can trigger a catastrophic debris cascade. CASCADE is engineered so that every uncertain, ambiguous, or computationally degraded object pair is *escalated* to a more expensive solver, never silently dropped. The result is a provable zero-false-negative property across the entire filter cascade.
- **Deterministic, auditable execution.** Identical inputs always produce the same maneuver sequence, the same CDM records, and the same fuel expenditure. This determinism is not incidental—it is enforced by 8 standalone CTest gate binaries that run on every commit, making the safety guarantee reproducible and CI-verifiable.
- **Operational realism.** The engine respects the physical constraints of real spacecraft operations: impulsive-burn propulsion with Tsiolkovsky fuel accounting, 600 s thruster cooldown, 15 m/s per-burn thrust limits, line-of-sight validation against a geographically distributed ground network, and a 10 s signal-latency budget that forces proactive burn scheduling before blackout windows.

The remainder of this report presents the system architecture (§2), our engineering rationale for key design choices (§3), the numerical methods underpinning propagation and screening (§4), the mission-console frontend (§5), validation evidence (§6), and the deployment pipeline (§7).

2. System Architecture

CASCADE is a single-process C++20 engine with a deliberately minimal dependency footprint: `cpp-httplib` v0.15.3 for the REST server, `simdjson` v3.9.4 for streaming JSON ingestion, and Boost headers for version verification—all managed

through CMake FetchContent with no system-level library requirements beyond the GCC 12 standard library and `libgomp` for OpenMP. This dependency discipline ensures that the build is fully reproducible from a clean Ubuntu 22.04 base image.

Figure 1 illustrates the data path traversed by every simulation tick, from telemetry ingestion through maneuver execution.

```
POST /api/telemetry
  ↓
[State Store] - lock-free SoA upsert
  ↓
[Propagation] - adaptive RK4+J2 (OpenMP)
  ↓
[Broad Phase] - O(N log N) SMA+incl bins
  ↓
[Narrow Phase] - MOID + TCA + RK4 refine
  ↓
[CDM Engine] - 24 h predictive scan
  ↓
[COLA Planner] - RTN evasion + Tsiolkovsky
  ↓
[Recovery] - CW/ZEM slot-targeting burn
  ↓
REST API + Frontend (port 8000)
```

Figure 1: End-to-end simulation tick pipeline. Each stage is executed sequentially within a single tick; the engine guarantees that all propagation completes before screening begins, and all screening completes before maneuver decisions are made.

2.1 Concurrency Model

The engine employs a *command-queue* concurrency pattern. Mutating operations (telemetry ingestion, simulation steps, maneuver scheduling) are serialized through a dedicated worker thread backed by `std::future`/`std::promise` pairs, ensuring that no two commands can interleave state modifications. Read-only endpoints (`GET /api/visualization/snapshot`, `GET /api/status`) operate concurrently via atomically published view pointers, avoiding lock contention during high-frequency frontend polling. Within a simulation step, computationally intensive loops—orbital propagation and narrow-phase refinement—are parallelized with OpenMP (`#pragma omp parallel for` with reduction clauses).

2.2 State Representation

All objects are stored in a Structure-of-Arrays (SoA) layout indexed by string identifier, pre-allocated for 10,100 objects (50 satellites + 10,000 debris + margin). Each record holds:

- 6-DOF ECI J2000 state vector $(\vec{r}, \vec{v})$ in km and km/s, with per-object telemetry epoch for asynchronous ingestion
- Precomputed Keplerian elements $(a, e, i, \Omega, \omega, M)$ derived via closed-form ECI-to-elements conversion at ingestion time
- Radial bounds $(r_{\min}, r_{\max})$ for shell overlap screening
- Fuel mass, cooldown timer, slot reference (bootstrapped from first telemetry and J2-secularly maintained), and full burn history

2.3 Telemetry Ingestion

Incoming telemetry arrives as a JSON payload containing an ISO-8601 timestamp and an array of objects, each with an identifier, type (`SATELLITE` or `DEBRIS`), and 3-component position and velocity vectors. The `simdjson` on-demand parser streams the payload through a pre-allocated padded buffer, avoiding intermediate copies. Invalid objects increment a diagnostic counter but never abort the batch—operational resilience demands that a single malformed record does not block ingestion of the remaining fleet. Parsed objects are applied atomically to the state store, and first-contact satellites trigger automatic Keplerian element computation and slot-reference bootstrapping.

2.4 API Surface

The engine exposes five primary endpoints on port 8000, bound to `0.0.0.0` as required by the problem statement:

Endpoint	Purpose
`POST /api/telemetry`	Batch ECI state ingestion
`POST /api/maneuver/schedule`	Evasion/recovery burn submission
`POST /api/simulate/step`	Advance simulation by Δt s
`GET /api/visualization/snapshot`	Geodetic snapshot for frontend
`GET /api/status`	Engine health, diagnostics, filter stats

Table 1: Primary REST API endpoints.

A rich set of debug endpoints (`/api/debug/{burns, conjunctions, propagation, conflicts}`) exposes internal ring buffers and per-tick statistics, providing full operational observability without additional logging infrastructure.

3. Engineering Rationale

Building a safety-critical orbital system under hackathon time constraints requires disciplined decision-making at every stage. Rather than committing early to a single architecture, we deliberately structured the development process around rapid prototyping, empirical evaluation, and principled selection—investing time upfront in exploring alternatives so that the final system rests on validated engineering judgments rather than assumptions. This

section documents the six most consequential decisions and the reasoning behind each.

3.1 Prototyping with Julia, Deploying in Pure C++

We recognized early that an orbital collision system involves two distinct workloads: (i) an *offline tuning* problem, where multi-objective optimizers sweep a large parameter space to find safe, efficient configurations, and (ii) a *real-time inference* problem, where the engine must execute a complete tick—propagation, screening, maneuver planning—within a deterministic latency budget. Julia's scientific computing ecosystem (`DifferentialEquations.jl`, native BLAS, first-class optimization libraries) made it the natural choice for prototyping the tuner, so we built a complete NSGA-II parameter optimizer via the `jluna` C++/Julia bridge, driving 69 engine parameters across safety, fuel, and latency objectives.

Once the tuner proved that the parameter space could be safely explored, we made a deliberate deployment decision: the production runtime would be pure C++20 with zero Julia dependency. The reasoning was threefold. First, the problem statement specifies a cold `docker build` on the grading VM, and the Julia 1.10 runtime adds ∼200 MB to the image plus 4–8 s of JIT warm-up—unacceptable for a latency-sensitive submission. Second, every numerical kernel the engine needs (RK4 integration, Keplerian solvers, CW state-transition matrices) admits a compact, dependency-free C++ implementation that compiles to tight machine code under `-O2`. Third, separating tuning from inference allowed us to guard the Julia bridge behind a CMake option (`PROJECTBONK_ENABLE_JULIA_RUNTIME`) and ship a minimal binary that links only against `libgomp1`—the OpenMP runtime—and the C++ standard library.

The offline tuner remains a first-class tool: its NSGA-II runs (20 population × 5 generations, 120 evaluations) produced the production defaults that the engine ships with, and all 69 parameters remain overridable at runtime via `PROJECTBONK_*` environment variables, enabling parameter sweeps without recompilation.

3.2 Staged Filter Activation via Shadow Deployment

Aerospace-grade conjunction screening systems face an inherent tension: aggressive filtering reduces computational cost, but every new filter introduces a risk of inadvertently discarding a true conjunction. In mission-critical contexts, releasing a new filter *directly into production* is analogous to deploying untested flight software—the consequences of a false negative are catastrophic.

We resolved this tension by adopting a *shadow deployment* discipline inspired by industry practices in fault-tolerant systems. Every new screening filter is introduced in three stages:

1. **Shadow mode**: The filter evaluates every candidate pair and logs its rejection decision, but the pair is *not* discarded. The full downstream pipeline processes the pair regardless.
2. **Calibration**: Shadow-mode telemetry—exposed via `/api/status?details=1`—accumulates rejection statistics against the ground-truth RK4 reference. The team reviews false-negative rates, threshold sensitivity, and edge-case behavior.
3. **Activation**: Only after the filter demonstrates zero false negatives across diverse orbital regimes (circular, eccentric, sun-synchronous, crossing) is it promoted from shadow to hard-reject mode via environment variable.

This pattern is currently applied to the D-criterion broad-phase gate, the plane/phase angular gate, and the MOID proxy. It guarantees that the zero-false-negative invariant is maintained at every point in the development timeline, while still allowing the team to develop and validate increasingly aggressive filters that will reduce narrow-phase compute by an estimated 40–60% upon activation.

3.3 Clohessy–Wiltshire Recovery with Heuristic Safeguard

Station recovery after an evasion maneuver is a classical relative-motion problem. We evaluated two approaches from the start: a proportional RTN gain controller (simple, robust, but producing suboptimal ΔV vectors on eccentric orbits) and a full Clohessy–Wiltshire (CW) state-transition inversion (mathematically optimal for near-circular relative motion, but singular at certain CW horizons).

The CW formulation computes the recovery impulse by inverting the in-plane 2×2 velocity-to-position submatrix of the full 6×6 state-transition matrix. We blend this with velocity-error damping:

$$\Delta\vec{v}_{\text{rec}} = \alpha\, \Phi_{rv}^{-1}\, \delta\vec{r} + \beta\, \delta\vec{v} \tag{1}$$

with $\alpha = 0.70$ (position correction weight) and $\beta = 0.30$ (velocity damping), tuned by the NSGA-II optimizer. The CW horizon is set to $0.25 \times T_{\text{orbit}}$, clamped to $[300, 5400]$ s. We further cap the CW correction at 15% of remaining ΔV budget and 40% of the heuristic norm to prevent over-correction—a nuance that matters when recovery burns occur near orbital resonances where the CW matrix condition number degrades.

The proportional controller is retained as an automatic fallback for the specific regime where the CW matrix becomes near-singular ($nt \approx k \cdot 2\pi$). This dual-solver architecture ensures that recovery planning never stalls, while defaulting to the mathematically superior solution in the vast majority of cases. The regression gate `recovery_slot_gate` enforces $\Delta_{\text{slot}} \leq 0.08$ deterministically across all test configurations.

3.4 Operational Dashboard over Visual Spectacle

The frontend design was driven by a careful reading of the problem statement's evaluation criteria. Section 7 allocates 15% weight to *UI/UX & Visualization* and specifically requires a ground track map, conjunction bullseye plot, telemetry heatmaps, and maneuver timeline—all inherently 2D operational surfaces. We recognized that the value proposition of the frontend lies in *situational awareness*: enabling a mission operator to understand fleet status, conjunction threats, and burn schedules at a glance.

Accordingly, we built the primary dashboard around Canvas 2D Mercator ground tracks with 90-minute historical trails and forecast paths, a polar "bullseye" threat-assessment view with radial time-to-closest-approach encoding, a scrollable Gantt maneuver timeline with blackout overlays, and per-satellite fuel gauges. A Three.js WebGL globe provides supplementary 3D spatial context, but it supports rather than dominates the interface. This architecture ensures that every pixel on screen serves an operational purpose aligned with the judge rubric, delivering the kind of mission-console experience that a real spacecraft operator would recognize. The current shell reinforces that principle by keeping shared controls compact: simulation stepping lives in the top command strip, while global focus and the operator checklist move into a collapsible sidebar so route-level canvases remain visually dominant.

3.5 Robust MOID Screening via Multi-Mode Solver

Minimum Orbit Intersection Distance (MOID) is a powerful pre-filter: if two orbits never come within a threshold distance of each other *anywhere* along their trajectories, no time-domain refinement is necessary. The challenge is that analytical MOID solvers—such as the Gronchi (2005) quartic formulation we implemented in `orbit_math.cpp`—are sensitive to orbital element degeneracies. Near-circular orbits (where the argument of perigee is undefined) and near-equatorial orbits (where RAAN is poorly determined) produce ill-conditioned systems.

Rather than restricting the analytical solver to well-conditioned regimes and leaving degenerate cases unscreened, we designed a *multi-mode* MOID architecture:

- **PROXY**: A 72×72 true-anomaly grid (5,184 evaluations per pair) that is robust to all orbit types, including circular and equatorial.
- **HF (High-Fidelity)**: Coarse global sampling at $2\times$ resolution followed by 3 iterations of local refinement (5×5 grid, halving step size), with pre-computed satellite orbit positions to amortize trigonometric evaluations.
- **ANALYTICAL**: The full ellipse-to-ellipse minimum distance computation for well-conditioned pairs.

The grid resolution (72 samples per orbit) was calibrated through analysis: at LEO altitudes, 5° angular resolution corresponds to ∼50 km arc length, providing reliable sub-kilometer MOID estimates. The 2.0 km reject threshold ensures that only genuinely non-intersecting orbit pairs are discarded. The PROXY mode is deployed in shadow mode, consistent with our staged activation discipline.

3.6 Provably Safe Fail-Open Escalation

When propagation fails for an object—producing NaN states, divergent trajectories, or eccentricities that exceed the valid domain—the engine must decide how to screen that object's conjunctions. An intuitive approach would be a tiered fallback: use last-known elements for a reduced broad-phase check. We formally analyzed this option and identified a critical flaw: if an object's orbital elements are stale or invalid, the broad phase cannot guarantee that its shell-overlap test captures all crossing-orbit debris. A stale semi-major axis could shift an object into the wrong SMA bin, causing it to miss pairs that a correct binning would catch.

The only strategy that *provably* maintains zero false negatives is to treat propagation-failed objects as maximally uncertain: the engine bypasses all cheap filters and checks the failed object against the *entire* debris catalog using the full narrow-phase pipeline. This is computationally expensive—$O(M)$ narrow-phase evaluations for each failed object—but the cost is bounded (propagation failures are rare in practice, typically affecting <0.1% of objects per tick) and the safety guarantee is absolute. Detailed fail-open counters are exposed in the engine's status API, providing full transparency into when and why this escalation path activates.

4. Numerical Methods

4.1 Orbital Propagation

Propagating 10,000+ objects per tick demands a scheme that is simultaneously accurate enough to detect 100 m-scale conjunctions and fast enough to fit within a deterministic latency budget. We designed an *adaptive three-mode* propagator that allocates computational effort in proportion to each object's orbital complexity.

Mode 1 — J2 Secular Fast Path. The vast majority of LEO objects occupy near-circular ($e \leq 0.02$), high-perigee (≥ 500 km) orbits where the dominant perturbation is Earth's oblateness. For these objects, we apply closed-form J2 secular rates to mean elements:

$$\dot{\Omega} = -\frac{3}{2} n J_2 \left(\frac{R_E}{p}\right)^2 \cos i \tag{2}$$

$$\dot{\omega} = \frac{3}{4} n J_2 \left(\frac{R_E}{p}\right)^2 (5\cos^2 i - 1) \tag{3}$$

where $n = \sqrt{\mu/a^3}$, $p = a(1-e^2)$, and $R_E = 6378.137$ km. An extended fast lane admits $e \leq 0.003$, perigee ≥ 650 km,

$\Delta t \leq 45\,\mathrm{s}$, covering the operating regime of most commercial constellations. Long horizons are sub-stepped at 300 s intervals to bound secular-rate linearity error.

Mode 2 — Probe-Then-Escalate. Objects that fall outside the fast-lane criteria first receive a J2-secular prediction, followed by a coarse RK4 probe (max step 120 s) that validates the result. If the disagreement exceeds 0.5 km in position or 0.5 m/s in velocity, the engine automatically escalates to full numerical integration. This two-stage check avoids paying the full RK4 cost for objects that are well-approximated by secular theory, while catching those that genuinely require numerical treatment.

Mode 3 — Full RK4+J2. Objects with low perigee ($< 350\,\mathrm{km}$), high eccentricity ($e \geq 0.98$), or long propagation horizons ($\Delta t > 21{,}600\,\mathrm{s}$) are integrated with classical fourth-order Runge–Kutta, maximum substep 60 s, under the J2-perturbed acceleration:

$$\vec{a}_{J2} = \frac{3}{2}\frac{J_2\,\mu\,R_E^2}{r^5}\begin{bmatrix} x(5z^2/r^2-1) \\ y(5z^2/r^2-1) \\ z(5z^2/r^2-3) \end{bmatrix} \tag{4}$$

Finite-value guards after every substep prevent NaN propagation; any divergent state triggers the fail-open escalation path described in §3.

Accuracy. Maximum position error at 86,400 s (a full orbit day) is $<$ **0.061 km**, verified by the regression gate `phase2_regression_gate`—approximately three orders of magnitude below the 100 m collision threshold.

Why RK4 over higher-order adaptive integrators. We deliberately chose classical RK4 over RKF4(5) or the Dormand–Prince pair. The collision threshold (100 m) is orders of magnitude larger than RK4 truncation error at a 60 s substep, so the additional accuracy of a higher-order method provides no safety benefit. More importantly, RK4 avoids the overhead of embedded error estimation and step-rejection logic, keeping the per-object cost *predictable*—a property that directly enables deterministic tick budgets. Our probe-then-escalate scheme already provides architectural-level adaptivity where it matters: allocating cheap secular propagation to easy objects and full numerical integration to hard ones.

4.2 Coordinate Transforms

All internal computation uses ECI J2000 coordinates. For visualization and ground-station LOS validation, the engine implements a full coordinate-transform chain:

- **ECI → ECEF**: Rotation about the z-axis by the Greenwich Mean Sidereal Time angle, computed from the standard polynomial expansion (IAU 2000 compatible).
- **ECEF → Geodetic**: Iterative Bowring algorithm on the WGS-84 ellipsoid ($a = 6378.137\,\mathrm{km}$, $f = 1/298.257223563$), converging to 10^{-13} rad in at most 8 iterations.
- **Geodetic → ECEF**: Prime vertical radius of curvature $N(\phi)$ with WGS-84 constants for ground station positions.
- **Local ENU**: East-North-Up frame at each ground station for elevation-angle computation (see §4.7).

4.3 Broad-Phase Candidate Generation

The broad phase reduces the $O(N^2)$ all-pairs space through three sequential geometric filters:

1. **SMA–Inclination Band Indexing.** Objects are binned by semi-major axis ($\Delta a = 500\,\mathrm{km}$) and inclination ($\Delta i = 20°$). Flat sorted arrays with binary search replace hash maps, exploiting cache locality on modern CPUs. Neighbor search extends ± 2 bins in each dimension to handle boundary-crossing orbits.
2. **Shell Overlap Test.** Perigee $r_{\min} = a(1-e)$ and apogee $r_{\max} = a(1+e)$ are compared with a 50 km margin. Objects with invalid elements fall back to an instantaneous radius $\pm 200\,\mathrm{km}$ envelope—an intentional over-approximation that preserves the zero-false-negative guarantee at the cost of passing more candidates to the narrow phase.
3. **D-Criterion Gate.** A simplified Drummond proximity metric:

$$D = \sqrt{(\Delta a/\bar{a})^2 + \Delta e^2 + (2\sin(\Delta i/2))^2} \tag{5}$$

with reject threshold $D > 2.0$. Deployed in shadow mode as part of the staged activation discipline (§3).

Fail-open path. Objects with invalid elements, $e > 0.2$, or radial span exceeding bin coverage bypass all cheap filters and proceed directly to the narrow phase. Detailed fail-open counters are exposed in `/api/status?details=1`, providing transparency into the fraction of the catalog that exercises this conservative path.

4.4 Narrow-Phase Collision Screening

Candidate pairs from the broad phase enter a multi-gate narrow pipeline designed to progressively concentrate computational effort on the highest-risk pairs:

1. **Plane/Phase Angular Gate.** Angular momentum vectors are computed from (i, Ω) and used to assess orbital plane alignment. Pairs with orbital plane angle $> 75°$ or mean anomaly phase separation $> 150°$ are rejected—their orbits cannot geometrically intersect within the screening horizon. Objects with $e > 0.2$ bypass this gate (fail-open), since their large radial excursion can bridge wide angular separations.
2. **MOID Proxy.** The multi-mode MOID solver (§3) evaluates minimum orbit-to-orbit distance. In the default PROXY mode, 5,184 sampled evaluations per pair provide sub-kilometer accuracy across all orbital regimes. Reject threshold: 2.0 km.

3. **Linear TCA Approximation.** Four simultaneous linear-segment minimum-distance estimates (start-velocity, end-velocity backward, averaged, and secant approximations) are computed over the step interval. The minimum d^2 across all four determines passage.
4. **RK4 Micro-Refinement.** Pairs within 0.10 km above the screening threshold receive a fine-grained RK4 propagation ($\Delta t_{\max} = 5\,\mathrm{s}$), resolving any ambiguity introduced by the linear TCA approximation.
5. **Full-Window RK4 Refinement.** Pairs within 0.20 km receive a budget-limited, OpenMP-parallelized full-window refinement (16 trajectory samples, 1 s substep). Thread-safe atomic budget decrement prevents compute runaway: budget tiers range from 20 (for >2M pairs) to 96 (<100K pairs), dynamically scaled by pair count and step size, clamped to $[8, 192]$.

Uncertainty promotion. Hypervelocity encounters (relative speed $> 8\,\mathrm{km/s}$) receive an additional 0.10 km screening margin, reflecting the reality that positional uncertainty grows faster at high closing speeds. The effective screening distance is:

$$d_{\text{screen}} = \underbrace{0.100}_{\text{collision}} + \underbrace{0.020}_{\text{TCA guard}} = 0.120\ \text{km} \tag{6}$$

4.5 24-Hour Predictive CDM Generation

After each simulation tick, the engine performs a forward-looking conjunction scan over a configurable horizon (default 86,400 s = 24 h). Broad-phase candidate pairs are independently RK4+J2 propagated from their post-step states in 600 s substeps (RK4 max internal step 30 s). For each time sample, the engine evaluates miss distance and approach speed, generating a *Conjunction Data Message* (CDM) record that includes satellite/debris identifiers, TCA epoch, miss distance, approach speed, both ECI positions, collision flag, and severity classification. Up to 1,024 predictive records are retained per scan, sorted by severity (CRITICAL first, then ascending miss distance), and exposed via the status and debug APIs.

Pairs where RK4 propagation fails—producing NaN or divergent states—are conservatively classified as CRITICAL, triggering the autonomous COLA pipeline. This ensures that computational failures never suppress collision warnings.

4.6 Autonomous Maneuver Planning

4.6.1 COLA Planner

When a predictive CDM record reaches CRITICAL severity ($d < 100\,\mathrm{m}$), the engine triggers an autonomous evasion burn search over a discrete candidate grid:

- 4 burn-time fractions of the remaining lead time
- 4 ΔV scales: $\{0.5, 1, 2, 4\} \times 0.001\,\mathrm{km/s}$
- 10 RTN directions: $\pm T$, $\pm R$, 4 diagonal T/R, $\pm N$

Each candidate burn is fully forward-propagated via RK4 with the impulse applied, and the resulting trajectory is screened against all active debris to compute the post-maneuver miss distance. The RTN frame is constructed using the true tangential direction:

$$\hat{r} = \frac{\vec{r}}{|\vec{r}|}, \quad \hat{n} = \frac{\vec{r} \times \vec{v}}{|\vec{r} \times \vec{v}|}, \quad \hat{t} = \hat{n} \times \hat{r} \tag{7}$$

Candidates are ranked by a lexicographic multi-criteria comparator: safety class (miss distance relative to 0.15 km target) $\rightarrow$ $|\Delta V|$ $\rightarrow$ normal component magnitude $\rightarrow$ slot error $\rightarrow$ predicted miss distance. A maximum of 3 CRITICAL CDMs are evaluated per satellite per tick, bounding the worst-case planning cost.

4.6.2 Fuel Accounting

Propellant expenditure follows the Tsiolkovsky rocket equation:

$$\Delta m = m_{\text{current}}\left(1 - e^{-|\Delta\vec{v}|/v_e}\right), \quad v_e = I_{\text{sp}} \cdot g_0 \tag{8}$$

with $I_{\text{sp}} = 300\,\mathrm{s}$, $g_0 = 9.80665\,\mathrm{m/s^2}$, $m_{\text{dry}} = 500\,\mathrm{kg}$, $m_{\text{fuel,init}} = 50\,\mathrm{kg}$. A 5% fuel guard (2.5 kg) triggers automatic graveyard orbit transfer, ensuring that end-of-life satellites are disposed of responsibly rather than becoming debris themselves.

4.6.3 Constraint Enforcement

The engine enforces the full set of operational constraints specified in the problem statement:

- **Cooldown**: 600 s mandatory rest between burns on the same satellite, preventing thruster thermal overload.
- **Thrust cap**: $|\Delta V| \leq 15\,\mathrm{m/s}$ per burn, reflecting realistic chemical thruster limits.
- **LOS validation**: Upload windows are scanned in 20 s steps across a 1,800 s horizon against a 6-station global ground network (§4.7). A 10 s signal latency is enforced.
- **Blackout handling**: Burns must be computed and queued before the satellite exits the last ground-station footprint, ensuring commands are uploaded in time.

4.7 Ground Network and Line-of-Sight

CASCADE models a globally distributed 6-station ground network, designed to maximize coverage of polar, equatorial, and mid-latitude LEO orbits:
For each candidate burn, the engine propagates the satellite's position to the planned upload epoch augmenting it with J2 secular rates, converts from ECI to ECEF via GMST rotation, transforms the satellite-to-station vector into the local ENU frame, and computes the elevation angle:

$$\theta_{\text{el}} = \arctan\left(\frac{u_{\text{Up}}}{\sqrt{u_{\text{East}}^2 + u_{\text{North}}^2}}\right) \tag{9}$$

If θ_{el} exceeds the station's minimum elevation mask at any point within the upload window, LOS is confirmed and the burn is eligible for scheduling. Station configurations are loaded from CSV at runtime

Station	Location	Min El.
ISTRAC Bengaluru	13.0° N, 77.5° E	5°
Svalbard Satellite	78.2° N, 15.4° E	5°
Goldstone Tracking	35.4° N, 116.9° W	10°
Punta Arenas	53.2° S, 70.9° W	5°
IIT Delhi Ground Node	28.5° N, 77.2° E	15°
McMurdo Station	77.8° S, 166.7° E	5°

Table 2: Ground station network. Elevation constraints are per-station to model real-world terrain and infrastructure masks.

(`PROJECTBONK_GROUND_STATIONS_CSV`), with a compiled-in fallback array ensuring graceful degradation if the file is unavailable.

4.8 Recovery Planning

Every evasion maneuver triggers a paired recovery burn computed via the CW/ZEM solver described in §3. The station-keeping box is defined as a 10 km spherical radius around the nominal orbital slot, which is bootstrapped from first telemetry and propagated with J2 secular rates. Recovery burns are conservatively budgeted: only 5% of the remaining ΔV allocation is consumed per recovery request, preventing over-correction cascades. Each planned recovery trajectory is re-simulated through the conjunction screening pipeline—a *ghost recovery* simulation—to ensure that the recovery burn itself does not create a new collision risk.

5. Mission Console Frontend

The frontend is a React 19 / TypeScript single-page application built with Vite 6 and rendered via Canvas 2D and Three.js WebGL. During live operation it polls the snapshot stream at 1 Hz and the status, burn, conjunction, and trajectory streams at 2 s cadence. The UI is organized around a shared tactical shell plus six purpose-built route views:

1. **Command / Mission Theatre** (`#/command`): Fleet-wide posture summary, shared mission-time context, a Three.js orbital globe for command geometry, and a right-hand mission rail for focus, threat, burn, and slot-integrity state. This is the default landing view and the operator's main situational-awareness surface.
2. **Ground Track** (`#/track`): Mercator-projection canvas rendering of all satellite positions with 90-minute historical trails and 90-minute forecast arcs, a live solar terminator overlay, ground-station markers with coverage footprints, and a persistent map legend with epoch timestamp. Geodetic coordinates are derived from the engine's ECI-to-geodetic transform chain.
3. **Threat Assessment** (`#/threat`): Polar "bullseye" plot centered on the selected satellite. Radial distance encodes time-to-closest-approach; angular position encodes the approach vector in the satellite's local frame. CDM severity determines marker color (green/yellow/red), while the encounter queue and detail rail expose the most urgent watch-lane events.
4. **Burn Operations** (`#/burn-ops`): Scrollable Gantt-style maneuver timeline with evasion and recovery burns, blackout-zone overlays, and cooldown friction markers. Per-satellite fuel gauges show remaining propellant and ΔV budget.
5. **Evasion Analysis** (`#/evasion`): Counterfactual burn comparison, clearance-gain versus fuel-spent analysis, and operator-facing explanation of the avoidance trade space.
6. **Fleet Status** (`#/fleet-status`): Constellation-level analytics—fuel heatmaps across the fleet, ΔV expenditure versus collisions avoided, station-keeping drift metrics, and satellite availability tracking.

Across these pages, the shell preserves a single cross-route focus model: pinning a spacecraft in one view updates the command globe, threat queue, track detail rail, and burn context everywhere else. This shared-state approach reduces duplicated controls while keeping route-level evidence aligned to the same selected vehicle.

The shared shell provides a persistent summary rail, a sticky truth and freshness banner, inline simulation-step controls, reasoning / spotlight / audio toggles, and a collapsible sidebar whose expanded utility panels house the global focus console and operator checklist. Snapshot, conjunction, burn, and trajectory endpoints jointly supply the route-level context consumed by these views. Canvas 2D drives the ground-track, bullseye, and Gantt surfaces; Three.js powers the 3D orbital theatre. The entire frontend is statically built, bundled into the Docker image, and served by `cpp-httplib`'s mount-point mechanism—no separate web server required.

6. Validation and Evidence

A safety-critical system requires validation that is as rigorous as the software it certifies. CASCADE's test infrastructure is designed around a principle borrowed from aerospace V&V practice: every safety invariant is backed by a standalone, deterministic gate that can be executed independently and whose pass/fail verdict is unambiguous.

6.1 Test Infrastructure

When configured with `BUILD_TESTING=ON`, the source tree registers 8 core CTest safety gates and builds the corresponding standalone binaries in `tools/`. Complementary shell harnesses in `scripts/` exercise the same invariants in packaged-runtime scenarios. The Dockerfile runs `ctest` inside the builder stage—if any registered gate fails, the image build itself fails—and GitHub Actions additionally runs the dedicated gate scripts plus the Docker smoke test on pushes and pull requests targeting `main`.

Gate	Enforced Invariant
`phase2_regression`	Propagator drift < 0.061 km at 86,400 s
`phase2_regression_short`	Strict-adaptive stability at 1 s / 120 s horizon
`broad_phase_validation`	Shell filter $\supseteq$ brute-force baseline
`narrow_phase_false_neg.`	Zero missed collisions vs. RK4 reference
`recovery_slot_gate`	Deterministic slot recovery
`recovery_planner_inv.`	CW/ZEM convergence, slot error ≤ 0.08
`maneuver_ops_invariants`	Cooldown, fuel, thrust-cap enforcement
`api_contract_gate`	PS-required API surface conforms to schema

Table 3: Core safety gates exercised through CTest / CI. Each gate is a binary verdict—pass or fail—with no subjective interpretation.

6.2 Real-Data Validation at Scale

Synthetic test suites verify correctness under controlled conditions, but real-world orbital catalogs exercise the engine against the full diversity of orbit types, element qualities, and edge cases that exist in the actual LEO population. To validate CASCADE under realistic conditions, we ingested a live Space-Track OMM catalog (`data.txt`, 34 MB) through the scenario generator (`tools/real_data_scenario_gen`).

The generator parsed 30,000+ objects via simdjson, filtered to LEO (periapsis $< 2{,}000$ km altitude), classified payloads as satellites and the remainder as debris, and ran simulation ticks at full scale. Results:

- **27,191** LEO objects loaded in < 1 s with zero ECI conversion failures
- 10 conjunctions detected against the real catalog at 2.1 s/tick (full 27K scale)
- All broad-phase filters correctly preserved pairs that the narrow phase subsequently confirmed
- Zero false negatives across the full catalog run

This real-data exercise also served as a stress test for the fail-open path: objects with poorly-determined elements in the Space-Track catalog were automatically escalated to the full narrow-phase pipeline, exercising the conservative screening path under production-representative conditions.

6.3 Safety Score Evidence

The problem statement weights safety at 25%, the highest of all six criteria, and imposes a severe penalty for even a single collision. CASCADE's safety evidence rests on three pillars:

- **Zero false negatives** across 6 independent test families: broad-phase sanity, narrow-phase canary, false-negative gate, calibration probe, regression, and maneuver-ops.
- **Transparent fail-open accounting**: Every object with uncertain or invalid elements is escalated to the full narrow-phase pipeline. Fail-open counters are tracked per tick and exposed in the status API, providing auditable evidence that no object is silently dropped.
- **Conservative screening margin**: Effective collision envelope of 0.120 km, with uncertainty promotion to 0.220 km for hypervelocity encounters (> 8 km/s relative speed).

6.4 Fuel Efficiency Evidence

- Tsiolkovsky-consistent fuel deduction validated against analytical expectations in `maneuver_ops_invariants_gate`.
- CW/ZEM recovery solver produces lower $|\Delta V|$ than the proportional heuristic across all tested orbit configurations.
- Evasion fuel and recovery fuel are tracked independently in burn summaries, enabling per-category ΔV accounting.
- Recovery budgets are capped at 5% of remaining ΔV per request, ensuring that no single over-correction event depletes the constellation's margin.

6.5 Algorithmic Speed Evidence

Scenario	Mean Tick	Scale
50 sats / 10,000 debris	13.3 ms	Baseline
100 sats / 20,000 debris	72.6 ms	2× stress
27,191 LEO objects (real)	2.1 s	Full Space-Track

Table 4: Tick latency benchmarks (`phase3_tick_benchmark`).

The broad phase reduces the candidate count from $O(N \cdot M)$ to a small fraction via SMA/inclination binning. OpenMP parallelization of the narrow-phase sweep yields a 31.5% speedup at 2× scale.

Hot-path optimizations. Profiling identified two critical bottlenecks. First, `std::pow(rmag, 5.0)` in the J2 acceleration was 10–50× slower than explicit multiplication (`r2*r2*rmag`); replacing it eliminated the dominant per-object cost. Second, the three J2 secular rate functions (§4) shared common subexpressions ($\cos i$, $\sin i$, $a^{3.5}$) that were being recomputed independently; factoring them into a precomputed struct removed redundant transcendental evaluations. Third, the MOID grid resolution was increased from 24 to 72 samples after analysis showed that 15° angular spacing left ∼1,780 km arc gaps at typical LEO altitudes, rendering the filter effectively inert.

6.6 Deployment Verification

The Docker API smoke gate (`scripts/docker_api_smoke.sh`) performs a fully

automated release-verification cycle:

1. Build Docker image from `Dockerfile`
2. Boot container on auto-selected port
3. Probe `/api/status`, POST telemetry, POST simulate/step, GET snapshot
4. Validate each response against expected schemas with Python assertions
5. Tear down container

This end-to-end gate catches regressions that unit tests alone cannot: image-build failures, port-binding issues, static-asset serving, and CORS configuration.

7. Deployment

7.1 Docker Build Architecture

CASCADE uses a 3-stage multi-stage Dockerfile designed for reproducibility, build speed, and minimal runtime footprint:

Stage	Base	Purpose
builder	`ubuntu:22.04`	GCC-12, CMake, sccache, CTest gates
frontend	`node:24-slim`	`npm ci` + Vite production build
runtime	`ubuntu:22.04`	Minimal: `libgomp1` + binary

Table 5: Docker build stages.

Several optimizations accelerate the build pipeline:

- **sccache**: Mozilla's distributed compiler cache (v0.8.1, static musl binary) is used as `CMAKE_C_COMPILER_LAUNCHER`, eliminating redundant recompilation across incremental builds.
- **Dependency pre-fetch**: A dedicated Docker layer copies only `CMakeLists.txt` and runs with `PROJECTBONK_PREFETCH_ONLY=ON`, downloading simdjson and cpp-httplib into a cache-mounted `_deps/` directory. Source changes therefore never re-trigger dependency downloads.
- **BuildKit cache mounts**: Four persistent caches (apt, `_deps`, build directory, sccache) survive across builds.
- **Build-time testing**: `ctest` runs inside the builder stage, meaning the Docker image *cannot be built* if any safety gate fails.

The runtime image runs as a non-root user (`bonk`), copies only the compiled binary, ground-station CSV, and the Vite-built frontend static assets. CORS is enabled via `PROJECTBONK_CORS_ENABLE=1` for frontend integration. The binary serves the frontend directly from `cpp-httplib`'s static mount point—no Nginx or reverse proxy required.

7.2 Quick Start

```
# Build and run (entire pipeline, including tests)
docker build -t cascade:local .
docker run --rm -p 8000:8000 cascade:local

# Verify the engine is healthy
curl -s http://localhost:8000/api/status \
  | python3 -m json.tool

# Full automated verification
./scripts/docker_api_smoke.sh
./scripts/run_ready_demo.sh
```

7.3 CI Pipeline

GitHub Actions runs a build-and-gate workflow on every push to `main` and every pull request targeting `main`:

1. Install toolchain and configure CMake (`gcc-12`, Release, `BUILD_TESTING=ON`)
2. Build submission targets and all core gate binaries
3. Run dedicated shell gate scripts for propagation, broad phase, narrow phase, recovery, maneuver ops, and API-contract validation
4. Run the CTest safety suite with `-output-on-failure`
5. Run the Docker API smoke gate, which rebuilds the packaged image and verifies backend plus bundled frontend startup

Any failure blocks the workflow, ensuring that revisions merged into `main` have already passed both source-level gate checks and the packaged-runtime smoke verification.

8. Engineering Scope and Future Extensions

Every engineering project involves deliberate scoping decisions. The following items represent conscious trade-offs we made to deliver a reliable, well-tested system within the hackathon timeline, along with the extensions we would pursue given additional time.

- **Perturbation model.** The engine models J2 oblateness, which dominates all other perturbations in LEO by 2–3 orders of magnitude over the problem statement's days-level prediction horizon. Atmospheric drag is applied heuristically for objects with perigee below 350 km. Lunar/solar third-body effects and solar radiation pressure are not currently modeled; incorporating them would improve accuracy at long horizons but adds complexity disproportionate to the benefit at the 24-hour CDM timescale.
- **Maneuver optimization.** The COLA planner explores a discrete 160-candidate space (4 magnitudes $\times$ 4 timings $\times$ 10 directions). A continuous nonlinear programming or sequential convex programming formulation would yield lower-ΔV evasion sequences, but the discrete search completes in < 1 ms, produces deterministic and fully auditable results, and—crucially—admits exhaustive safety verification of every candidate. The same determinism that enables reproducible CI gates would be harder to maintain with a continuous optimizer.
- **Shadow-mode filter activation.** The D-criterion, plane/phase, and MOID gates currently operate in shadow mode, accumulating calibration evidence. Activating them would reduce narrow-phase compute by an estimated 40–60% with no impact on safety,

once the calibration data confirms zero false negatives across adversarial edge cases. The infrastructure for activation is fully built—it requires only flipping an environment variable per filter.

- **Dense-scene rendering.** Frontend rendering at 10,000+ debris markers has been verified in development; formal 60 FPS proof under worst-case load is an outstanding validation item. The Canvas 2D rendering path is inherently scalable (no DOM nodes per marker), so we expect this to be confirmed with a targeted benchmark.

Acknowledgments

We thank the organizers of the National Space Hackathon 2026 at IIT Delhi for designing a technically rigorous and operationally grounded problem statement that pushed us to engage deeply with the real challenges of autonomous spacecraft operations. Space-Track.org provided the real OMM catalog used for full-scale validation. The GCC, simdjson, cpp-httplib, React, Three.js, and Vite open-source communities provided the foundational tools on which CASCADE was built.

A. Key Physical Constants

Symbol	Description	Value
μ	Earth grav. parameter	$398\,600.4418\,\mathrm{km}^3/\mathrm{s}^2$
R_E	Earth equatorial radius	$6\,378.137\,\mathrm{km}$
J_2	Oblateness coefficient	1.08263×10^{-3}
d_{coll}	Collision threshold	$0.100\,\mathrm{km}$
I_{sp}	Specific impulse	$300\,\mathrm{s}$
g_0	Standard gravity	$9.80665\,\mathrm{m/s}^2$
m_{dry}	Satellite dry mass	$500\,\mathrm{kg}$
m_{fuel}	Initial propellant	$50\,\mathrm{kg}$
$\lvert\Delta V\rvert_{\max}$	Thrust limit	$15\,\mathrm{m/s}$
τ_{cd}	Cooldown period	$600\,\mathrm{s}$
m_{EOL}	Fuel EOL guard	$2.5\,\mathrm{kg}$
τ_{sig}	Signal latency	$10\,\mathrm{s}$
r_{box}	Station-keeping box	$10\,\mathrm{km}$

Table 6: System constants (from `src/types.hpp`).

B. CDM Severity Tiers

Tier	Condition	Auto-COLA
CRITICAL	$d < 0.100\,\mathrm{km}$	Yes
WARNING	$d < 1.0\,\mathrm{km}$	No (monitor)
WATCH	$d < 5.0\,\mathrm{km}$	No (log)

Table 7: Conjunction severity classification.

C. Verification Commands

```
# Build (local)
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build --target ProjectBONK

# Run all safety gates
ctest --test-dir build --output-on-failure

# Individual gates
./scripts/api_contract_gate.sh ./build
./scripts/broad_phase_sanity_gate.sh
./scripts/narrow_phase_false_negative_gate.sh
./scripts/recovery_planner_invariants_gate.sh

# Docker release verification
./scripts/docker_api_smoke.sh

# Frontend build
cd frontend && npm ci && npm run build

# Ready demo
./scripts/run_ready_demo.sh
```

D. NSGA-II Offline Tuner

Configuring a multi-tier screening pipeline with 69 free parameters requires more than manual trial-and-error. We invested in a principled offline optimization tool: a full NSGA-II (Deb et al., 2002) multi-objective evolutionary optimizer implemented in Julia and bridged to the C++ engine via `jluna`. The optimizer simultaneously minimizes three objectives:

1. **Safety risk** (number of false negatives; any > 0 renders the solution infeasible via feasibility-first constraint handling)
2. **Fuel consumption** (total ΔV across the constellation over the evaluation scenario)
3. **Tick latency** (wall-clock time for one simulation step)

The 69 parameters span three tiers:

- **Tier 1 — Broad phase** (9 params): SMA bin width, inclination bin width, shell margin, neighbor search range, D-criterion threshold, fail-open eccentricity cutoff.
- **Tier 2 — Narrow phase** (21 params): plane/phase angular thresholds, MOID grid resolution and reject threshold, TCA guard margin, micro-refinement band, full-refine budget tiers, uncertainty promotion velocity cutoff.
- **Tier 3 — Maneuver planner** (39 params): COLA ΔV scales, RTN direction weights, CW horizon bounds, position/velocity correction blend, recovery budget cap, CDM scanner substep and horizon.

A representative run (20 population $\times$ 5 generations = 120 evaluations) confirmed that all Pareto-optimal candidates maintained safety_risk = 0, with the best tick latency at 13.3 ms versus 23 ms under naïve defaults—a 42% improvement with zero safety compromise. The tuner's output became the production defaults that the engine ships with. All 69 parameters remain overridable at runtime via `PROJECTBONK_*` environment variables, enabling rapid experimentation without recompilation.

E. Problem Statement Alignment

Table 8 maps each PS §7 evaluation criterion to the CASCADE subsystem and evidence that addresses it.

Criterion	Wt.	CASCADE Evidence
Safety Score	25%	Zero false negatives (8 gates), fail-open escalation, conservative screening margin
Fuel Efficiency	20%	Tsiolkovsky accounting, CW/ZEM recovery, per-burn ΔV tracking
Constellation Uptime	15%	10 km slot box, J2 slot propagation, recovery_slot_gate (≤ 0.08)
Algorithmic Speed	15%	13.3 ms baseline tick, $O(N \log N)$ broad phase, OpenMP narrow phase
UI/UX & Viz.	15%	6-page dashboard, shared tactical shell, ground track, bullseye, Gantt timeline
Code Quality	10%	8 CTest gates, CI enforcement, Docker smoke, env-override system, debug APIs

Table 8: PS evaluation criteria alignment.

F. Development Timeline

CASCADE was developed over approximately 4 weeks across 60+ commits. The timeline below reflects a deliberate progression from foundational infrastructure to validated, deployable software.

Phase	Milestone
Week 1	Core infrastructure: telemetry ingestion via simdjson, SoA state store, J2 propagator with adaptive mode selection, Kepler solver, REST API skeleton on port 8000
Week 2	Screening pipeline: conservative broad phase (shell overlap, D-criterion), multi-gate narrow phase (TCA, MOID, RK4 refinement), CW/ZEM recovery solver, Tsiolkovsky fuel accounting, cooldown and thrust-cap enforcement
Week 3	Optimization and validation: shadow-gate observability, NSGA-II tuner (69 params), real-data scenario generator (27K Space-Track objects), OpenMP parallelization, environment-variable override system, 8 CTest safety gates
Week 4	Integration and polish: React/Canvas/WebGL dashboard (6 pages), operational dashboard design, 3-stage Docker build with sccache, CI pipeline, PS alignment analysis, technical report

Table 9: Development timeline. Each week built upon the validated outputs of the prior phase.